

Conjunto de instrucciones del RISC-V

Instrucciones de cómputo con constantes

Diseño de Sistemas Electrónicos

Felipe Machado

Instrucciones tipo R

	Tipo R	
<i>Add</i>	add	Aritméticas
<i>Subtract</i>	sub	
<i>AND</i>	and	A nivel de bits
<i>OR</i>	or	
<i>Exclusive OR</i>	xor	
<i>Set if Less Than</i>	slt	Comparaciones
<i>Set if Less Than Unsigned</i>	sltu	
<i>Shift Left Logic</i>	sll	Desplazamientos
<i>Shift Right Logic</i>	srl	
<i>Shift Right Arithmetic</i>	sra	

Instrucciones tipo R

Tipo R

	funct7	rs2	rs1	funct3	rd	opcode
add	0 0 0 0 0 0 0	rs2	rs1	0 0 0	rd	0 1 1 0 0 1 1
sub	0 1 0 0 0 0 0	rs2	rs1	0 0 0	rd	0 1 1 0 0 1 1
and	0 0 0 0 0 0 0	rs2	rs1	1 1 1	rd	0 1 1 0 0 1 1
or	0 0 0 0 0 0 0	rs2	rs1	1 1 0	rd	0 1 1 0 0 1 1
xor	0 0 0 0 0 0 0	rs2	rs1	1 0 0	rd	0 1 1 0 0 1 1
slt	0 0 0 0 0 0 0	rs2	rs1	0 1 0	rd	0 1 1 0 0 1 1
sltu	0 0 0 0 0 0 0	rs2	rs1	0 1 1	rd	0 1 1 0 0 1 1
sll	0 0 0 0 0 0 0	rs2	rs1	0 0 1	rd	0 1 1 0 0 1 1
srl	0 0 0 0 0 0 0	rs2	rs1	1 0 1	rd	0 1 1 0 0 1 1
sra	0 1 0 0 0 0 0	rs2	rs1	1 0 1	rd	0 1 1 0 0 1 1

Operaciones con constantes (inmediatos)

Suma de registro y constante

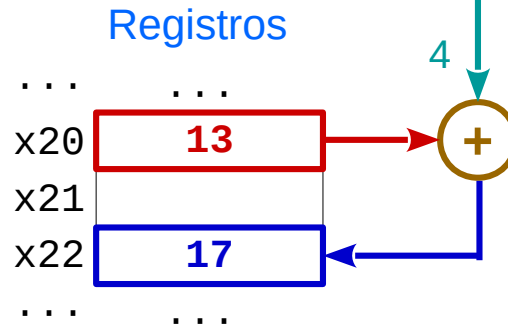
```
addi rd, rs1, imm #rd ← rs1 + imm
```

Registros

Constante (inmediato)

Ejemplo

```
addi x22, x20, 4 #x22 ← x20 + 4
```



Es muy habitual utilizar constantes en las sumas

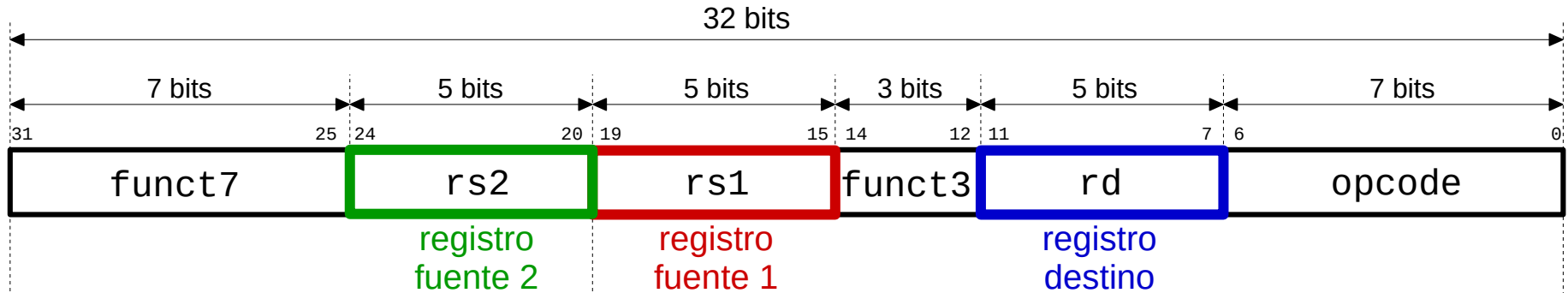
Este tipo de operaciones tiene 2 registros y una constante en la propia instrucción

Hacer que el caso más frecuente sea rápido

Operaciones con constantes

add rd, rs1, rs2

Formato instrucción tipo R



addi rd, rs1, imm

Formato instrucción tipo I

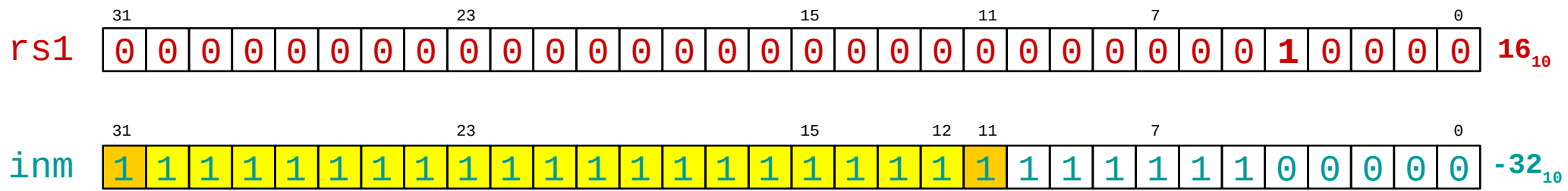
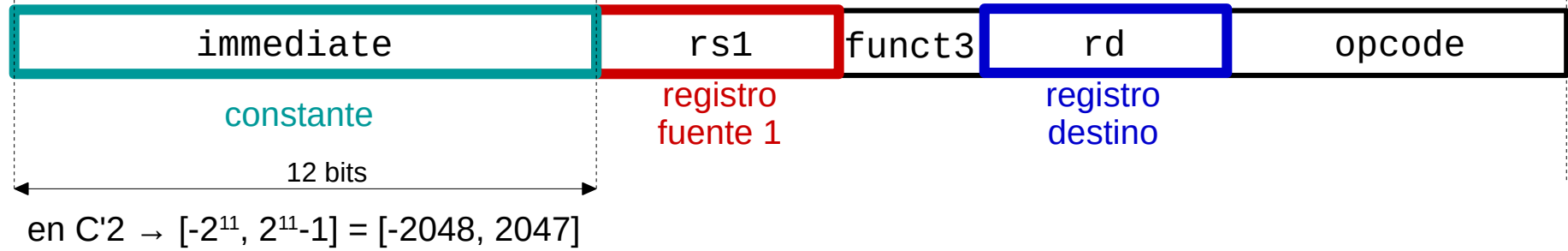


$C'2 \rightarrow [-2^{11}, 2^{11}-1] = [-2048, 2047]$

Suma constante

addi x19, x21, -32

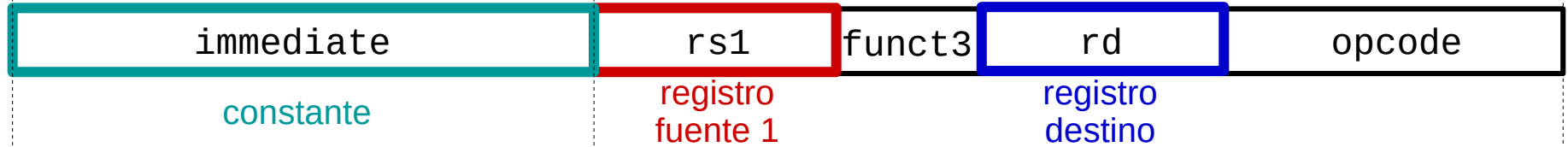
Formato instrucción tipo I



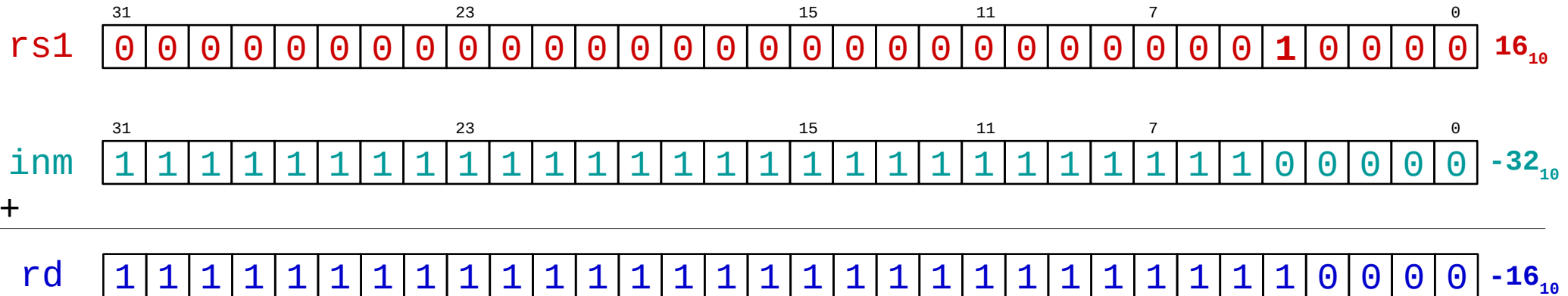
Suma constante

addi x19, x21, -32

Formato instrucción tipo I



en C'2 $\rightarrow [-2^{11}, 2^{11}-1] = [-2048, 2047]$



Suma constante

Lenguaje C



Ensamblador

```
i = i + 1;
```

```
addi x19, x19, 1
```

```
i++
```

Lenguaje C



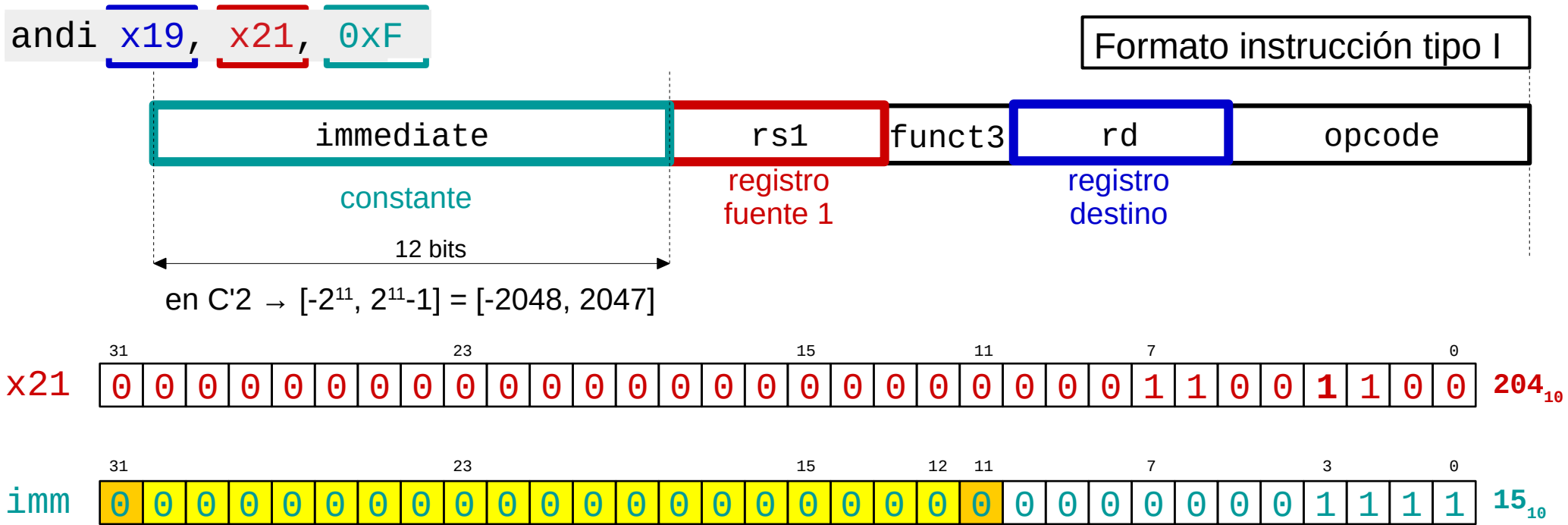
Ensamblador

```
i = 5;
```

```
addi x19, x0, 5
```

*Hacer que el caso más
frecuente sea rápido*

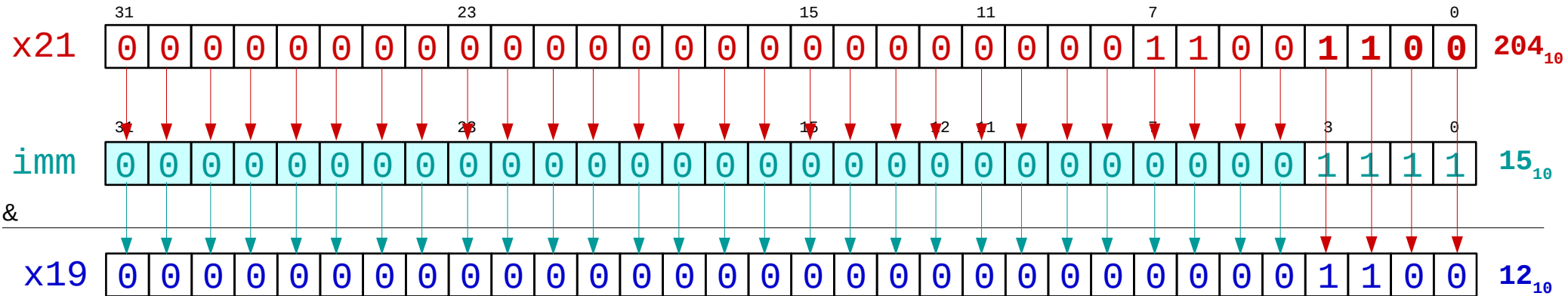
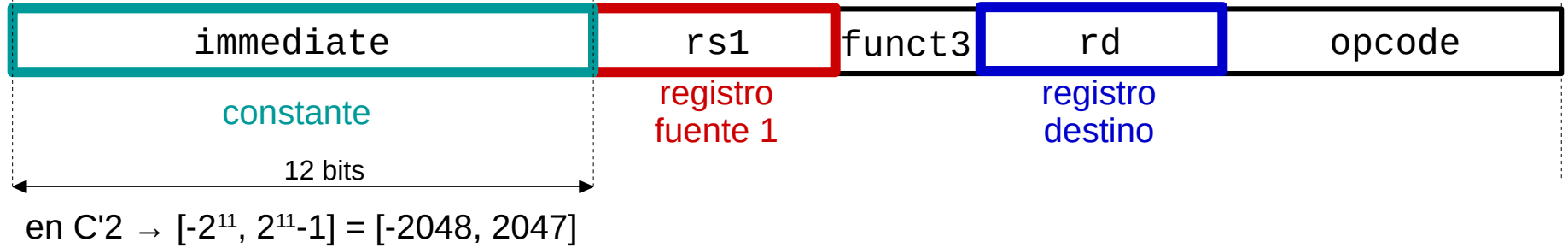
AND con constante



AND con constante

andi x19, x21, 0xF

Formato instrucción tipo I



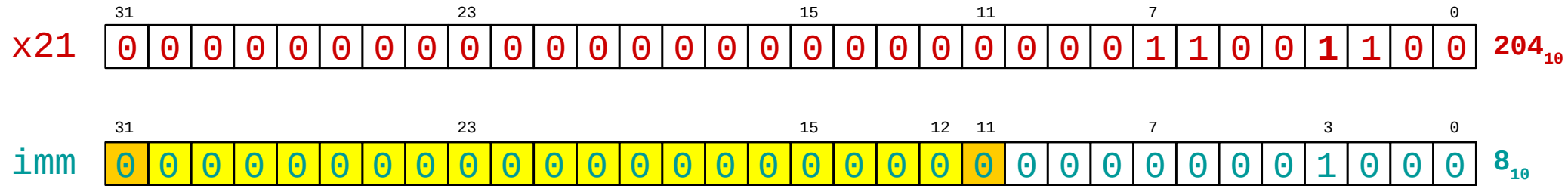
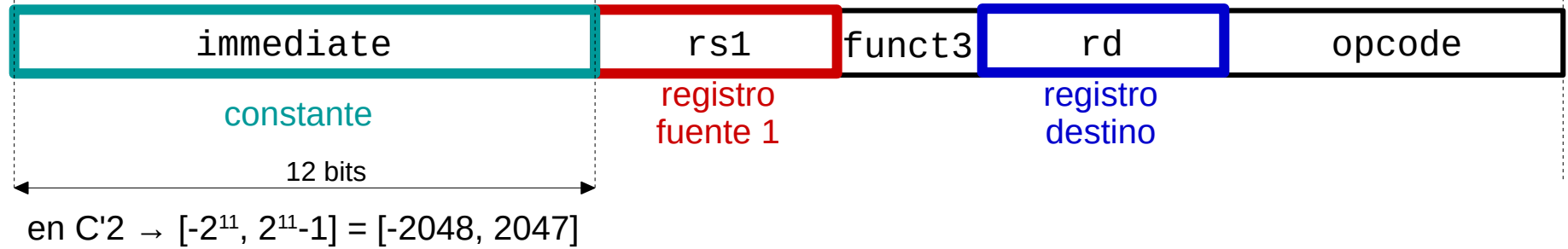
Operaciones con constantes

¿Cómo saber si el bit 3 de un registro vale 0 ó 1?

AND con constante

andi x19, x21, 8

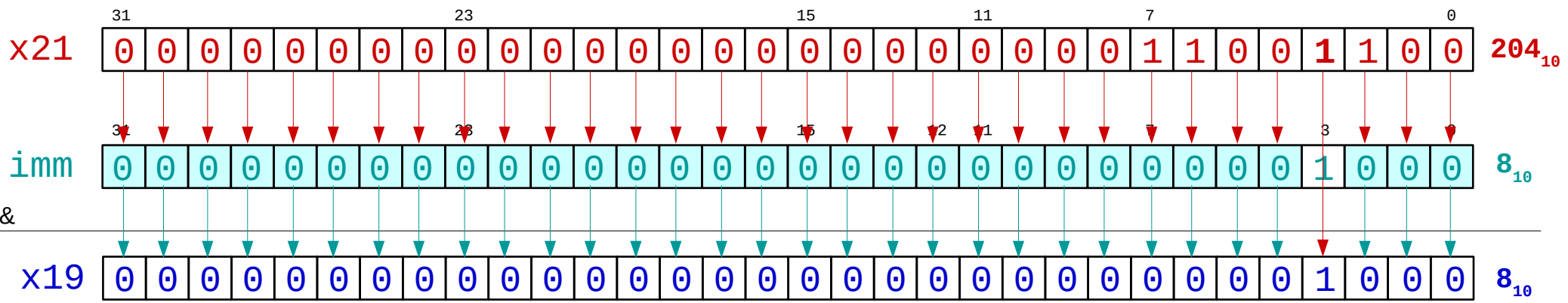
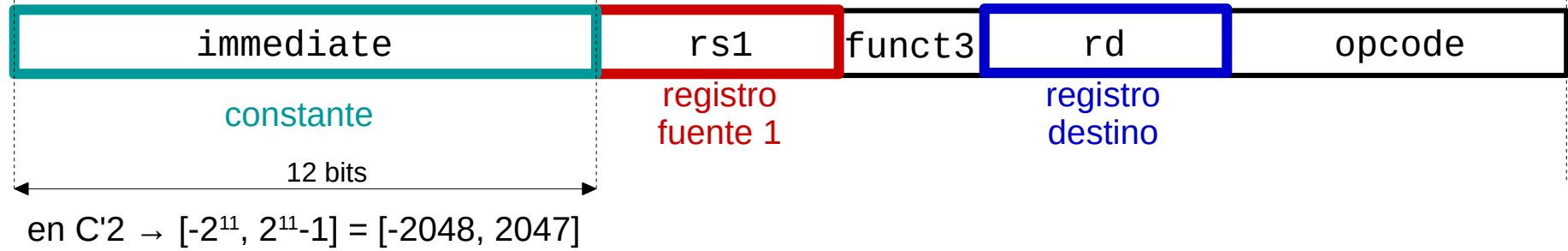
Formato instrucción tipo I



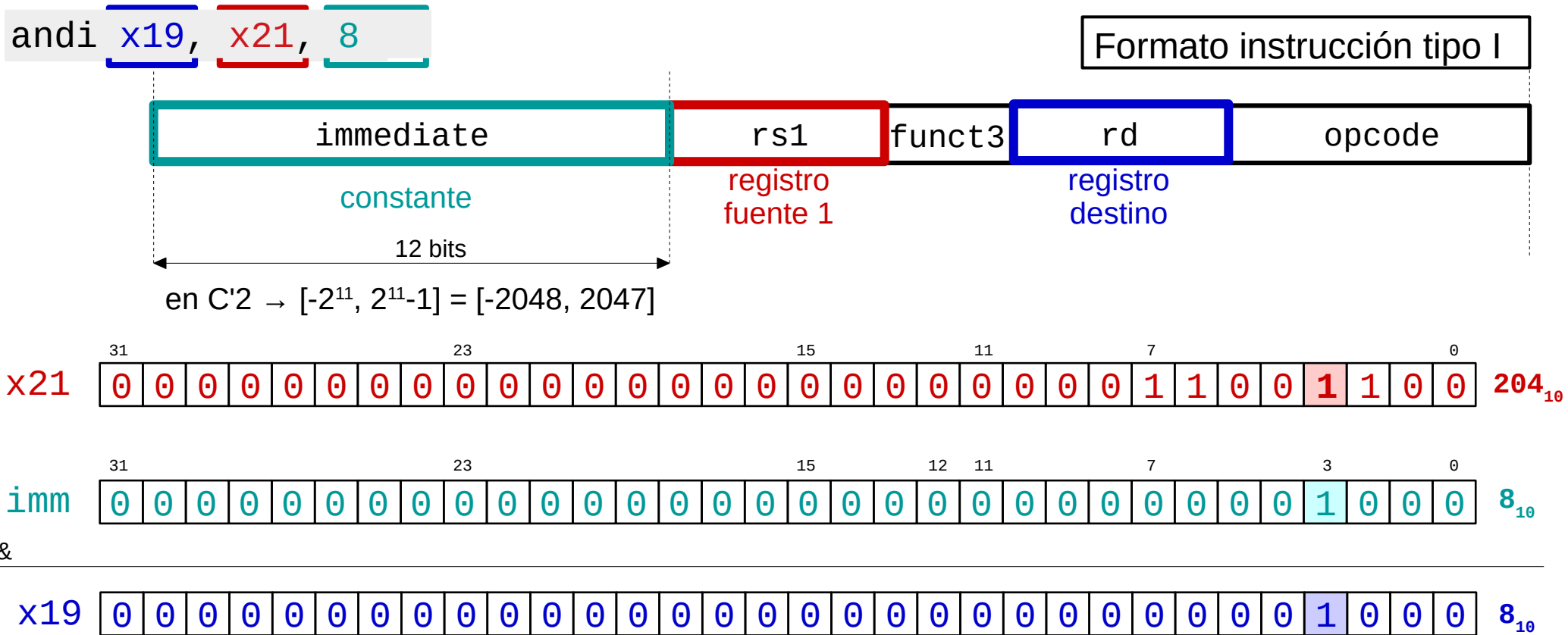
AND con constante

andi x19, x21, 8

Formato instrucción tipo I

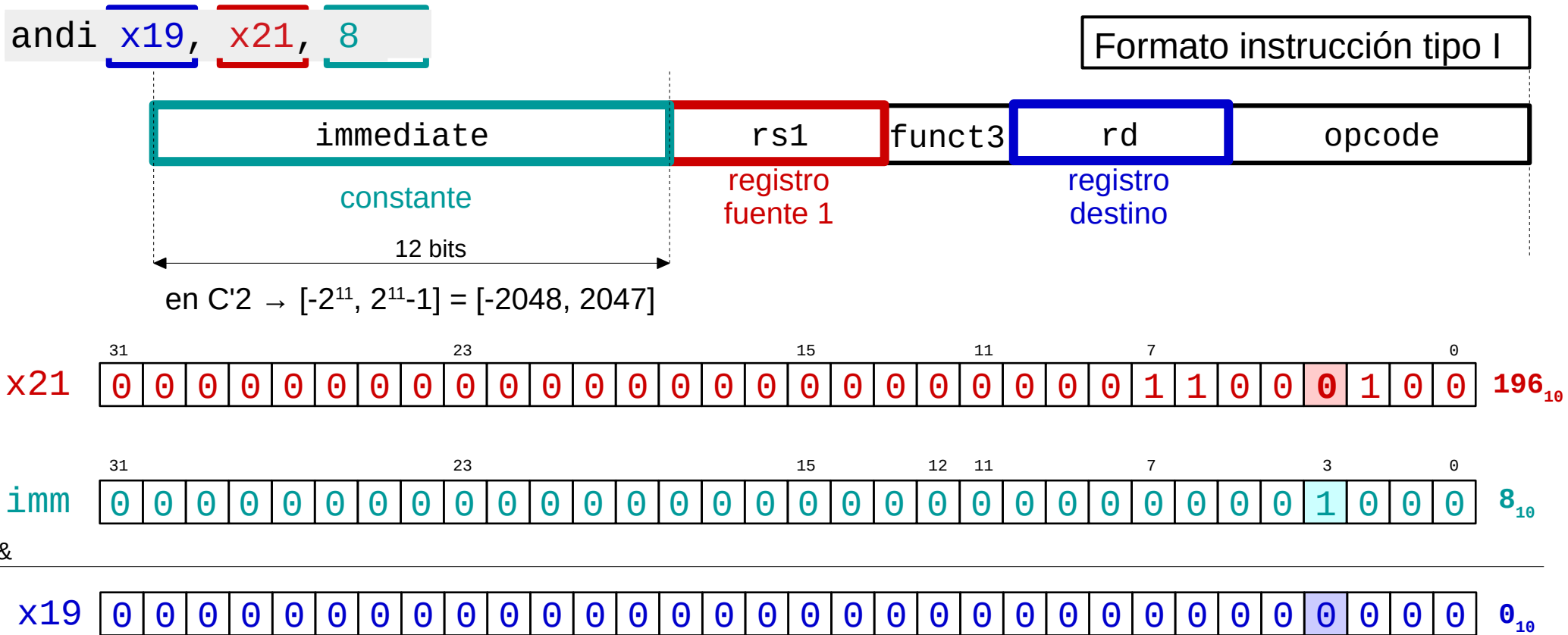


AND con constante



Si x19 es $> 0 \rightarrow$ el bit 3 vale 1 si no, vale 0

AND con constante



Si x19 es $> 0 \rightarrow$ el bit 3 vale 1 si no, vale 0

AND con constante

¿Cómo saber si el bit 3 del registro 21 vale 0 ó 1?

Ensamblador

```
andi x19, x21, 8
```

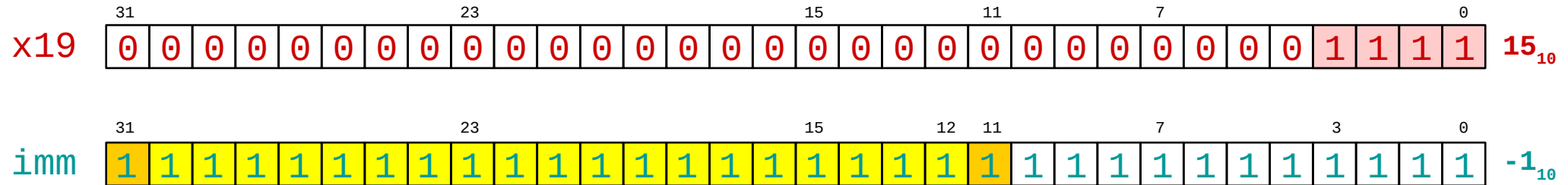
Si $x19 == 0 \rightarrow$ el bit 3 vale 0, si no, el bit 3 vale 1

XOR con constante

¿Cómo invertir los bits de un registro?

Ensamblador

```
xori x19, x19, -1
```

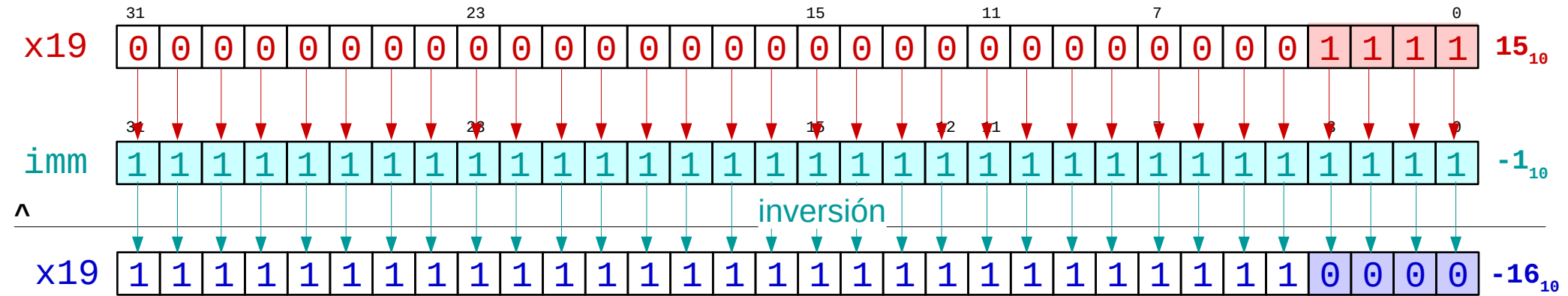


XOR con constante

¿Cómo invertir los bits de un registro?

Ensamblador

```
xori x19, x19, -1
```



Operaciones con constantes

¿Cómo invertir los bits de un registro?

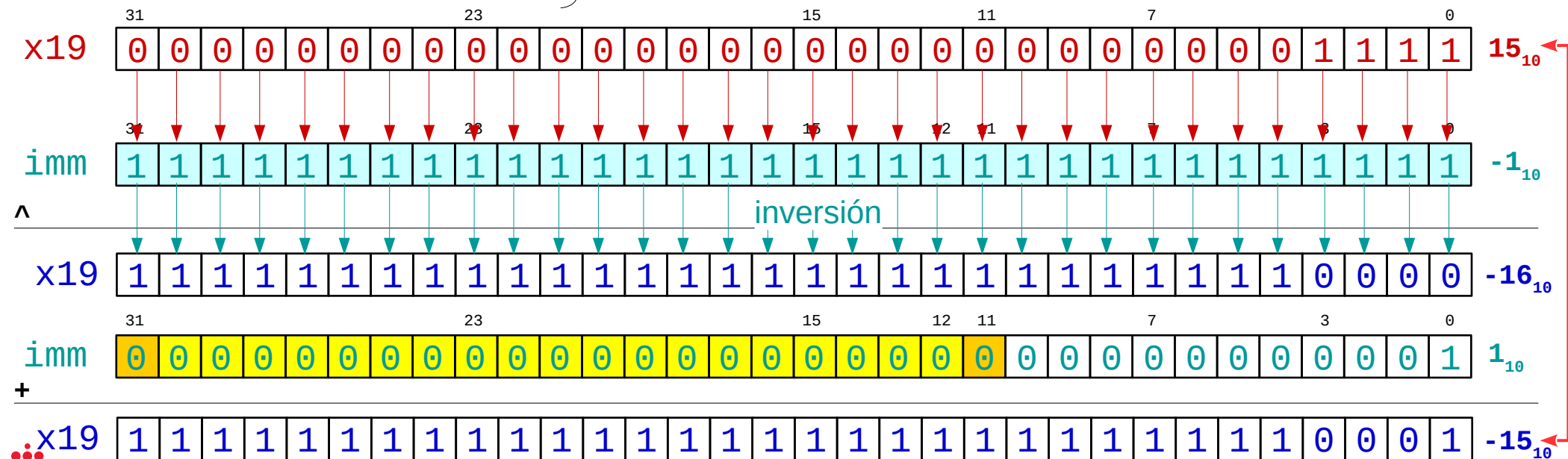
Ensamblador

```
xori x19, x19, -1
```

```
addi x19, x19, 1
```

negar el número:

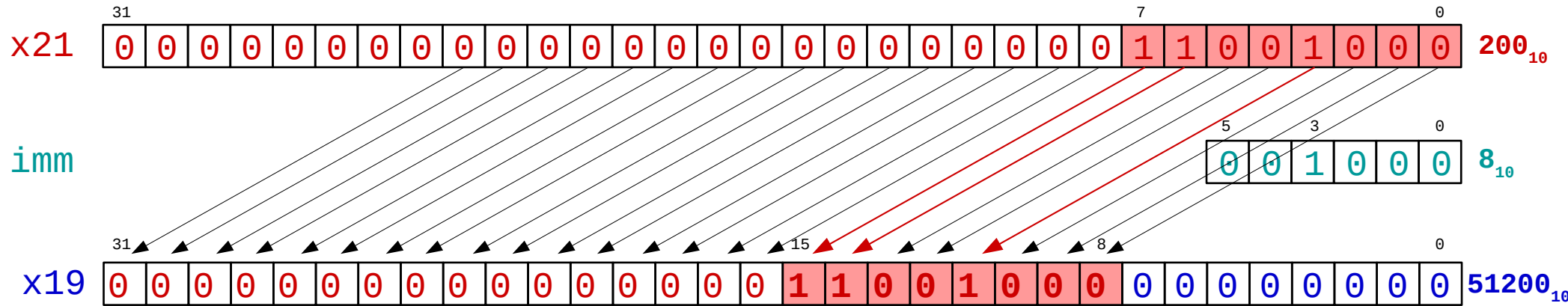
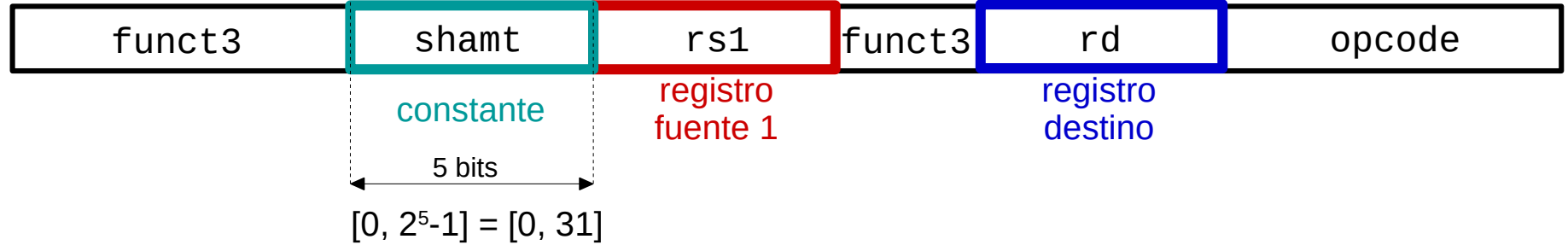
```
sub x19, x0, x19
```



Desplaza con constantes

slli x19, x21, 8

Formato instrucción tipo I



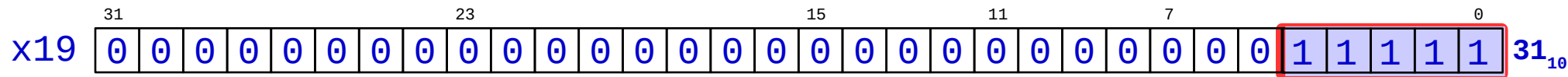
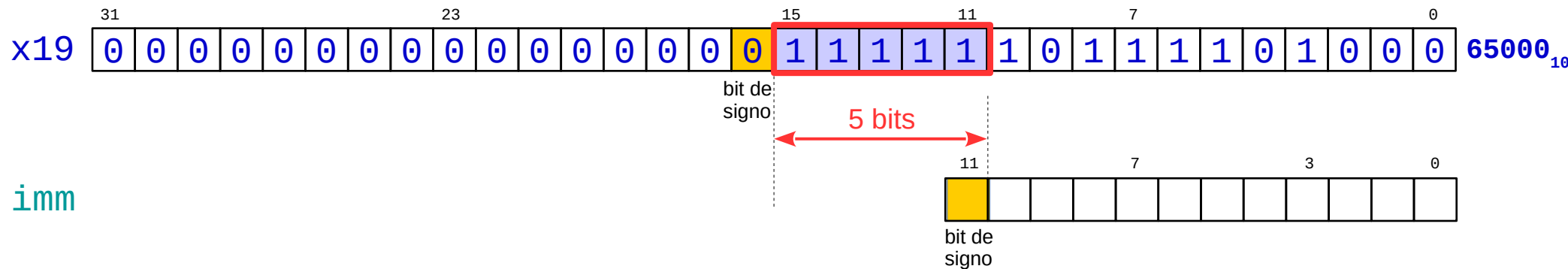
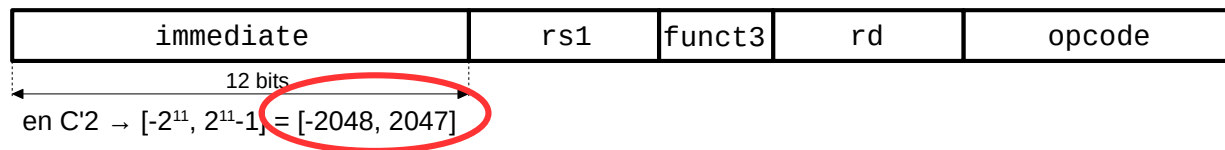
$$200 \cdot 2^8 = 200 \cdot 256 = 51200$$

Operaciones con constantes

Lenguaje C

```
i = 65000;
```

➡ 65000 > 2047

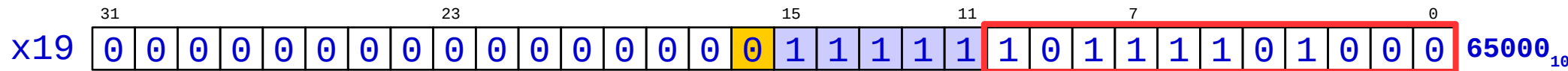
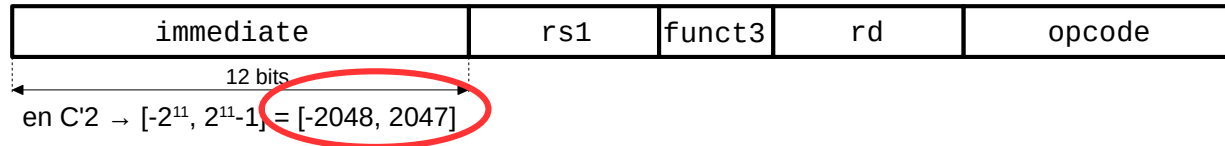


```
addi x19, x0, 31
```

Operaciones con constantes

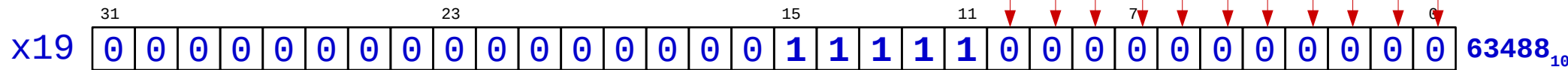
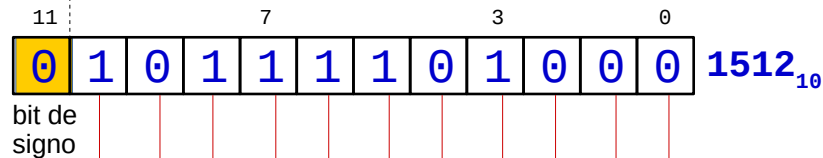
Lenguaje C

```
i = 65000;
```


$$65000 > 2047$$


bit de
signo

5 bits



```
addi x19, x0, 31
```

```
slli x19, x19, 11
```

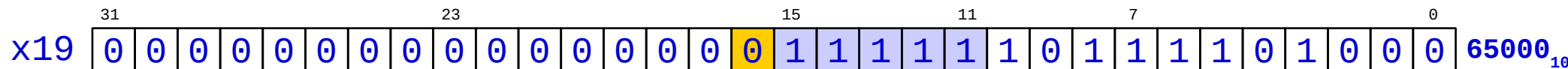
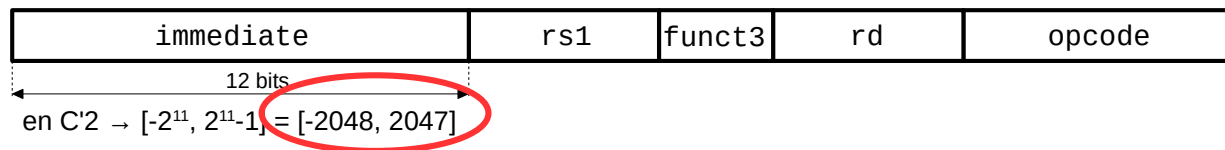
$$31 \cdot 2^{11} = 31 \cdot 2048 = 63488$$

```
addi x19, x19, 1512
```

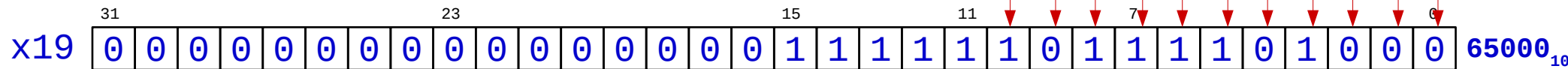
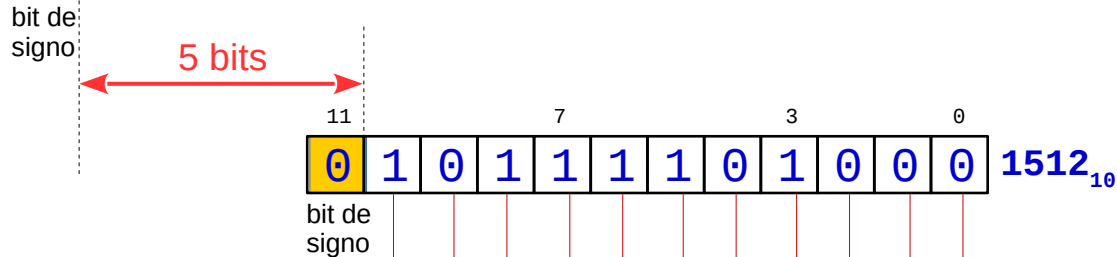
Operaciones con constantes

Lenguaje C

```
i = 65000;
```


$$65000 > 2047$$


imm



```
addi x19, x0, 31
```

```
slli x19, x19, 11
```

$$31 \cdot 2^{11} = 31 \cdot 2048 = 63488$$

```
addi x19, x19, 1512
```

Instrucciones tipo R e I

	Tipo R	Tipo I	
<i>Add</i>	add	addi	Aritméticas
<i>Subtract</i>	sub	—	
<i>AND</i>	and	andi	A nivel de bits
<i>OR</i>	or	ori	
<i>Exclusive OR</i>	xor	xori	
<i>Set if Less Than</i>	slt	slti	Comparaciones
<i>Set if Less Than Unsigned</i>	sltu	sltiu	
<i>Shift Left Logic</i>	sll	slli	Desplazamientos
<i>Shift Right Logic</i>	srl	srli	
<i>Shift Right Arithmetic</i>	sra	srai	

Instrucciones tipo I (inmediato)

Tipo I

addi	imm(11:0)								rs1	0 0 0	rd	0 0 1 0 0 1 1
andi	imm(11:0)								rs1	1 1 1	rd	0 0 1 0 0 1 1
ori	imm(11:0)								rs1	1 1 0	rd	0 0 1 0 0 1 1
xori	imm(11:0)								rs1	1 0 0	rd	0 0 1 0 0 1 1
slti	imm(11:0)								rs1	0 1 0	rd	0 0 1 0 0 1 1
sltiu	imm(11:0)								rs1	0 1 1	rd	0 0 1 0 0 1 1
slli	0 0 0 0 0 0 0	shamt	rs1	0 0 1	rd	0 0 1 0 0 1 1						
srli	0 0 0 0 0 0 0	shamt	rs1	1 0 1	rd	0 0 1 0 0 1 1						
srai	0 1 0 0 0 0 0	shamt	rs1	1 0 1	rd	0 0 1 0 0 1 1						

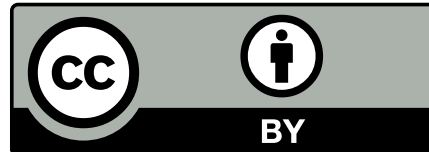
312524201915141211760

5 bits → desplazan los 32 bits del registro

diferencia con
instrucciones tipo R

Bibliografía y referencias

- Guía Práctica de RISC-V: El Atlas de una Arquitectura Abierta
David Patterson, Andrew Waterman
<http://riscvbook.com/espanol/>
- Digital Design and Computer Architecture
David Money Harris, Sarah L. Harris
- Computer Organization and Design. RISC-V Edition
David Patterson, John Hennessy
- The RISC-V Instruction Set Manual.
Vol I: User-Level ISA. v2.2
Andrew Waterman, Krste Asanovic
<https://riscv.org/specifications/>
- Design of the RISC-V Instruction Set Architecture
Andrew Waterman
<http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-1.html>
- <https://riscv.org/>
- <https://github.com/kvakil/venus>



Este tutorial:

Instrucciones de cómputo con constantes
Conjunto de instrucciones del RISC-V

cuyo autor es Felipe Machado

está bajo una licencia de Creative Commons Reconocimiento 4.0 Internacional

<https://creativecommons.org/licenses/by/4.0/>

doi:[10.5281/zenodo.4422986](https://doi.org/10.5281/zenodo.4422986)

versión 1.0

Video disponible: <https://youtu.be/22iIFspcYBM>



Universidad
Rey Juan Carlos

Área de Tecnología Electrónica

Felipe Machado